Lecture 8: Fork/Exec and XV6 Scheduling

Professor G. Sandoval

Some slides adapted by G. Sandoval for CS3224, from Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.
Also some Slides by Brendan Dolan-Gavitt and Junfeng Yang @ Columbia.

From last time

- Processes
- Process states
- Zombie state

Today

- Fork/Exec
- Scheduling
- XV6 Scheduling

Fork()

- System Call that creates an exact copy of current process called the child process.
- The child is initially a copy of the parent, but can be used to run a different branch of the program.
- After forking child and parent processes run in parallel.
- Any variables local to the parent process would have been copied to the child process so updating the variable on one process would not affect the other.

Fork() uses

- Google Chrome uses fork to handle each page within a separate process. This prevents client side code from bringing down the whole browser.
- Your shell uses fork() to run the programs you invoke from the command line.
- Apache web server uses fork() to create multiple server processes to handle requests.
- In general fork() is used to spawn processes in some parallel programs. Note this is different from threads since processes don't share the address space.

Fork example

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>


int main(void) {
    int num = 10;
    int pid = fork();

    if( pid > 0){
        printf("I'm the parent: My child has a pid: %d\n", pid);
        pid = wait(NULL);
        printf("My child is done\n");

    }
    else if (pid == 0) {
        printf("I am the child: exiting\n");
        exit(0);
    }
    return 0;
}
```

Fork example 2

```
int counter = 0;
pid_t pid = fork();

if (pid == 0)
{
    // child process
    int i = 0;
    for (; i < 5; ++i)
    {
        printf("child process: counter=%d\n", ++counter);
    }
}
else if (pid > 0)
{
    // parent process
    int j = 0;
    for (; j < 5; ++j)
    {
        printf("parent process: counter=%d\n", ++counter);
    }
}
```

Process Control Block in xv6

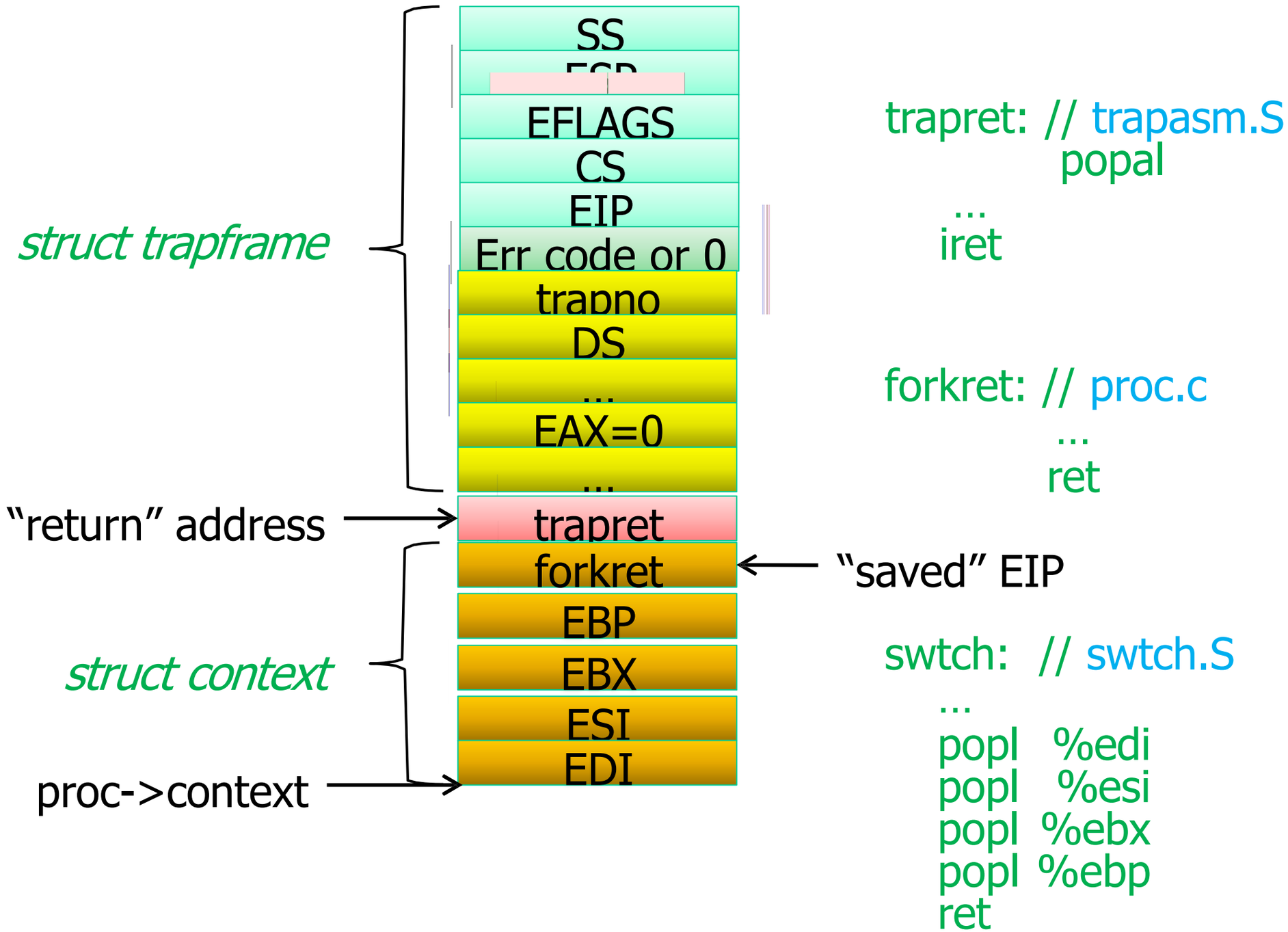
```
enum procstate { UNUSED, EMBRYO, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  uint sz;                     // Size of process memory (bytes)
  pde_t* pgdir;                // Page table
  char *kstack;                // Bottom of kernel stack for this process
  enum procstate state;        // Process state
  int pid;                     // Process ID
  struct proc *parent;         // Parent process
  struct trapframe *tf;        // Trap frame for current syscall
  struct context *context;     // swtch() here to run process
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)
};
```

fork() Implementation

- sysproc.c, proc.c
- Allocate new PCB and stack
  - Set up EIP of child to forkret -> trapret
- Copy address space
  - Copy both page tables and physical pages
- Set parent pointer
- Copy parent's trap frame
- Change EAX in trap frame so that child returns 0
- Copy open file table

Child process's kernel stack



```c
// Create a new process copying p as the parent.
// Sets up stack to return as if from system call.
// Caller must set state of returned proc to RUNNABLE.
int
fork(void)
{
  [...]
  // Allocate process.
  if((np = allocproc()) == 0)
    return -1;

  // Copy process state from p.
  np->pgdir = copyuvm(proc->pgdir, proc->sz)) == 0)
  [...]

  np->sz = proc->sz;
  np->parent = proc;
  *np->tf = *proc->tf;

  // Clear %eax so that fork returns 0 in the child.
  np->tf->eax = 0;

  [...]

  return pid;
}
```

XV6 fork Impl.

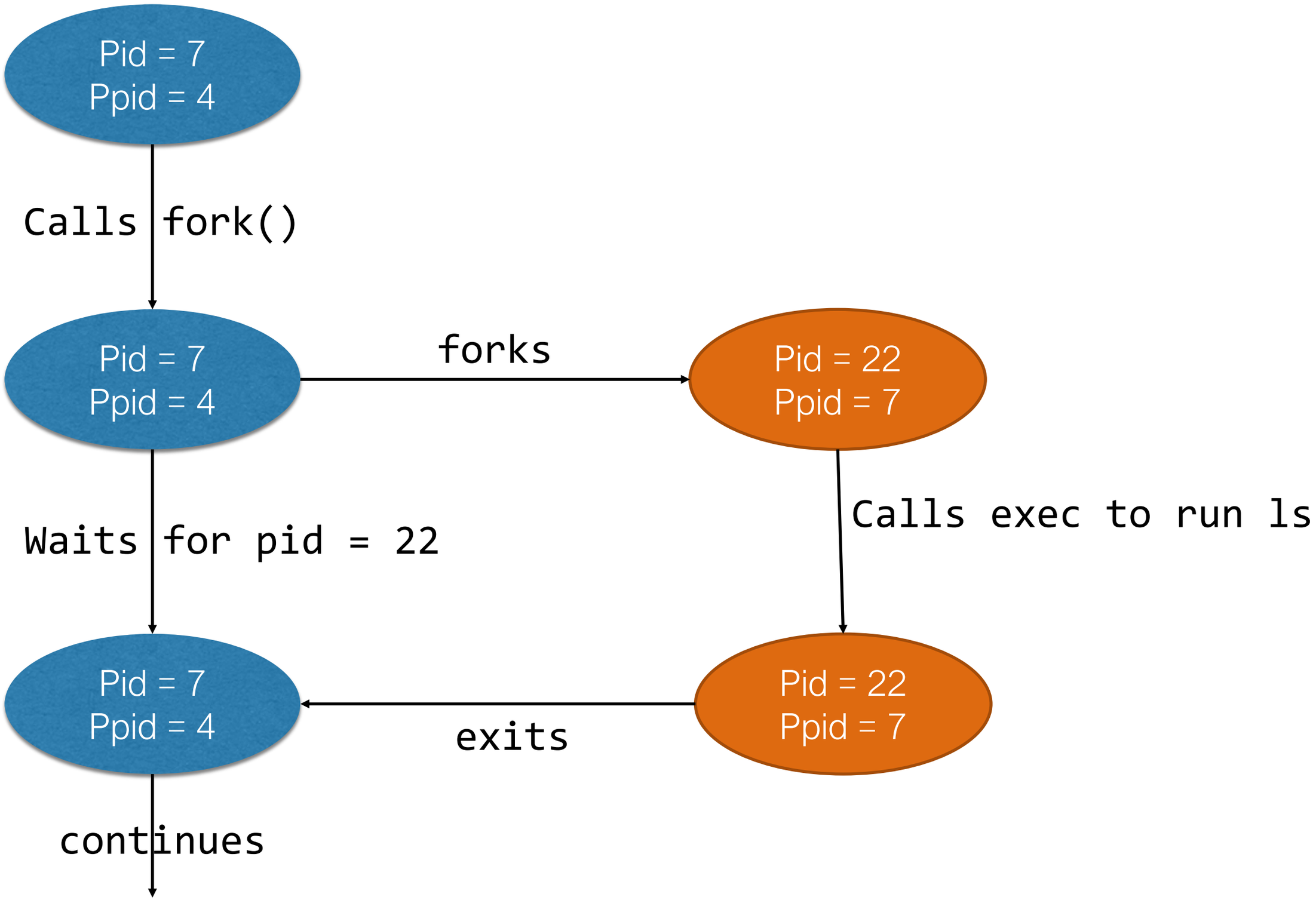
The Role of Exec

- If all we had was fork(), we couldn't launch new programs (why?)
- So we also have exec(), which allows you to *replace* the code and data in the current process with new code and data
- In modern UNIX systems, exec has many variants that provide different features:
  - Passing a variable number of arguments
  - Searching the directories listed in the PATH environment variable for the program

Bash example

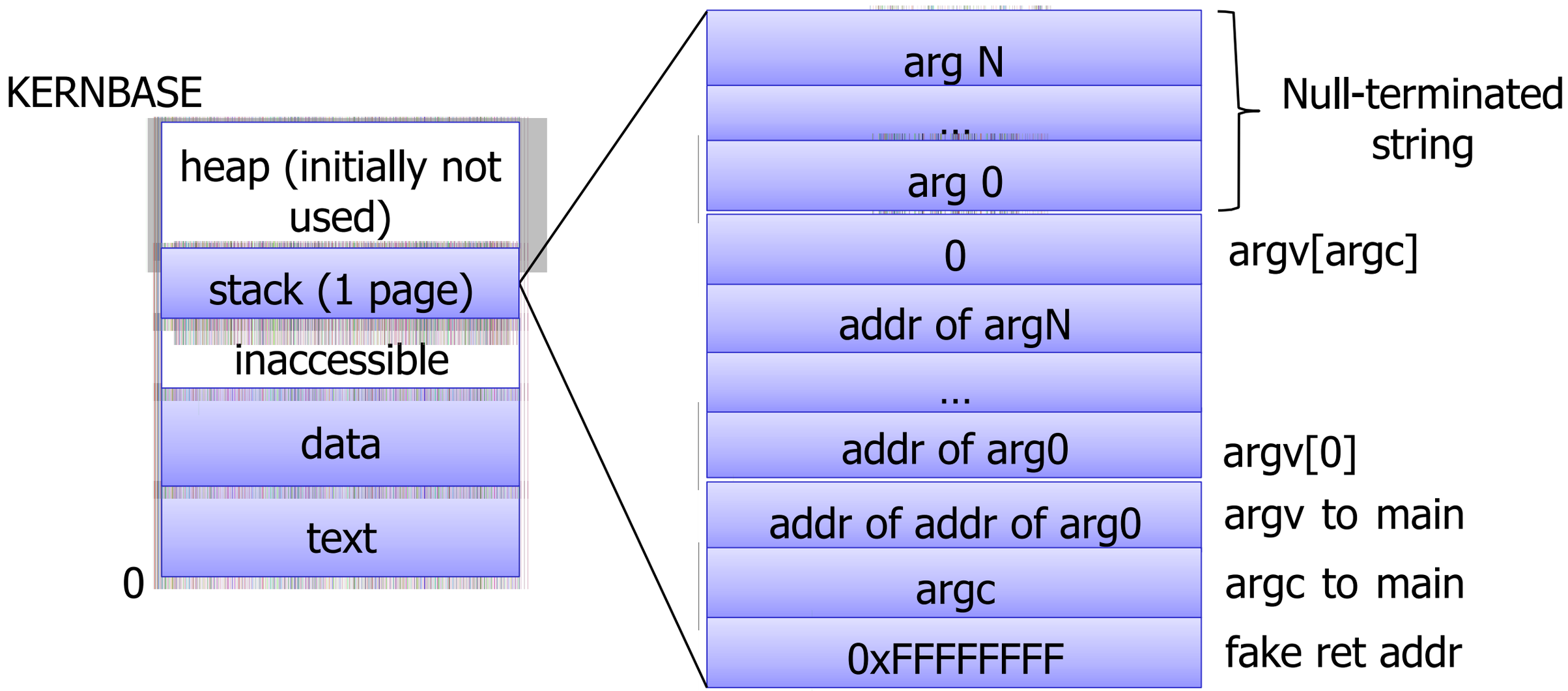
- How would you use fork/exec in a shell?
- For example when the shell calls `ls()`

Bash shell example



exec() Implementation

- sysfile.c, exec.c
- Set up user page table
- Load segments of the executable file into memory
- Set up stack and arguments to main(int argc, char* argv[])
- Jump to entry point (main) of the executable



In-Class Exercise

How many times

is "foo" printed?

```c
#include <stdio.h>
#include <unistd.h>

int main(void) {
    int i = 0;
    for (i = 0; i < 4; i++) {
        fork();
        printf("foo\n");
    }

    return 0;
}
```

Today

- Fork/Exec
- Scheduling
- XV6 Scheduling

My understanding of fork() and exec() is ?

Powered by Poll Everywhere

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Scheduling

- To coordinate the running of multiple processes, the OS includes a *scheduling algorithm* that decides what process will be run when, and for how long
- There are tons of different scheduling algorithms with different goals:
  - Maximize CPU utilization
  - Maximize I/O throughput
  - Make the system feel responsive to the user
  - Meet real-time deadlines

Other Scheduling Considerations

- May want different process *priorities*
- May want to handle I/O bound vs CPU-bound processes differently
- May want to ensure *fairness* – make sure each process gets time to run

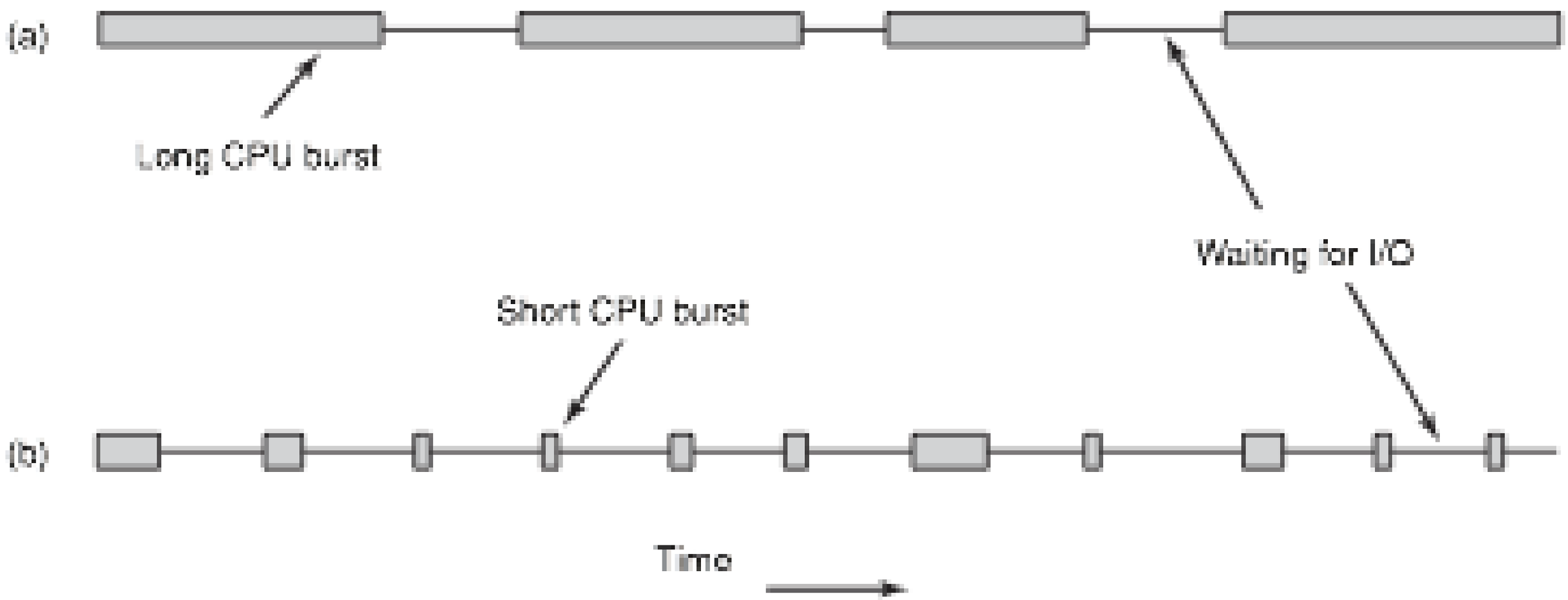


Figure 2-39. Bursts of CPU usage alternate with periods of waiting for I/O. (a) A CPU-bound process. (b) An I/O-bound process.

When to Schedule

- At timed intervals
  - We can use the hardware *timer interrupt* for this
- When a process exits
  - No process is running now so we must choose one
- When a process blocks on I/O
- When an interrupt happens
  - May signal that I/O is done, and we want to unblock the relevant process

Preemption

- Schedulers can be further classified by whether or not processes can choose when they stop running
- Non-Preemptive schedulers let processes run until they block on I/O or yield voluntarily
- Preemptive schedulers take advantage of a timer interrupt

Round Robin Scheduling

- A simple, *preemptive* scheduling algorithm
- Run first process until its quantum is used up
- Move that process to the end and run the next process until its quantum is used up
- Simple, fair

Multiplexing

- An operating system is likely to run with more processes than there are processors!
- OS provides the illusion that it has it's own (virtual) processor
- The OS does this by multiplexing multiple virtual processor on a single physical processor

Multiplexing

- XV6 multiplexes by switching each processor from one process to another in 2 situations:
  - Sleep and wakeup mechanism – wait for device or pipe (I/O)
  - Periodically forces switch when quantum ends

Multiplexing Challenges

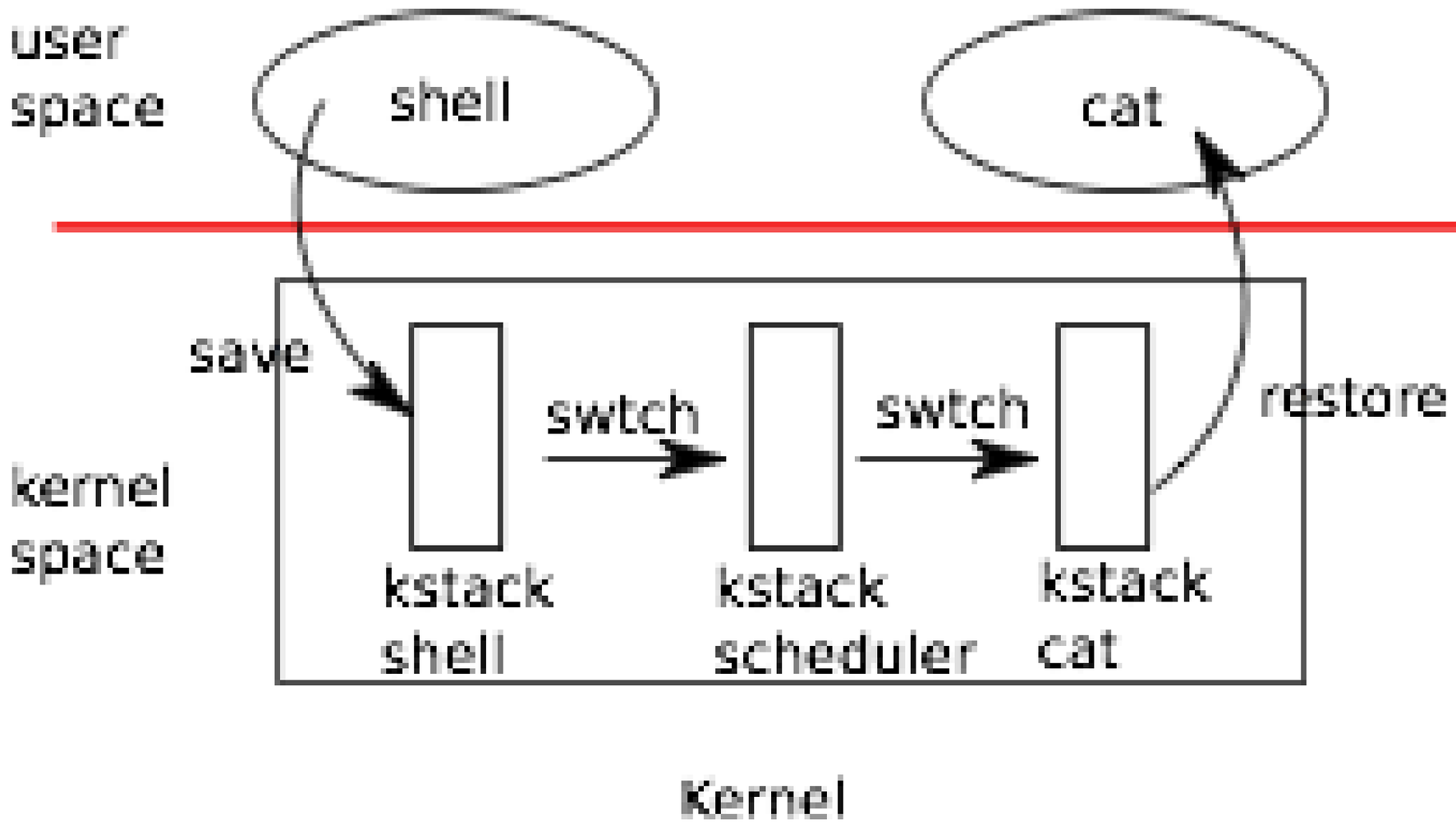
1. How to switch from one process to another? Implementation is tricky!
2. How to do context switching transparently?
3. Third how to handle multiple CPUs and avoid race conditions
4. Free memory after exit, but careful with kernel memory.

Today

- Fork/Exec
- Scheduling
- XV6 Scheduling

xv6 Scheduler

- xv6 uses *round robin*
- Whenever scheduling happens, loop over the array until we find a runnable process and switch to it
- When the timer runs or the process yields, we return to the scheduler loop
- Importantly, we return to the loop at the exact place we left off – this is what makes it *fair* and ensures we don't starve processes later in the list


user
space
shell
cat
save
swtch
swtch
restore
kernel
space
kstack
shell
kstack
scheduler
kstack
cat
Kernel

Proc.c

```
Void scheduler(void)
{
  struct proc *p;

  for(;;){
    // Enable interrupts on this processor.
    sti();

    // Loop over process table looking for process to run.
    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->state != RUNNABLE)
        continue;

      // Switch to chosen process.  It is the process's job
      // to release ptable.lock and then reacquire it
      // before jumping back to us.
      proc = p;
      switchuvm(p);
      p->state = RUNNING;
      swtch(&cpu->scheduler, proc->context);
      switchkvm();

      // Process is done running for now.
      // It should have changed its p->state before coming back.
      proc = 0;
    }
    release(&ptable.lock);
[. ..
```

switches between Kernel & process.

Context Switching

- The actual scheduler context switch (to start executing the process) happens here:
- Let's examine these in more detail

```
// Switch to chosen process.  It is the process's job
// to release ptable.lock and then reacquire it
// before jumping back to us.
proc = p;
switchuvm(p);
p->state = RUNNING;
swtch(&cpu->scheduler, proc->context);
switchkvm();
```

switchuvm

- This function does two things:
  - Switches the *task state segment* to the user-mode one
  - Changes the current *virtual address space* to the process's

Task State Segment (TSS)

- The TSS is a special structure defined by x86 intended to help with *context switching*
- *It provides:*
  - Location of kernel stack for *this process*
  - Segment descriptor for kernel stack

31	15	0	
I/O Map Base Address	Reserved	T	100
Reserved	LDT Segment Selector		96
Reserved	GS		92
Reserved	FS		88
Reserved	DS		84
Reserved	SS		80
Reserved	CS		76
Reserved	ES		72
EDI			68
ESI			64
EBP			60
ESP			56
EBX			52
EDX			48
ECX			44
EAX			40
EFLAGS			36
EIP			32
CR3 (PDBR)			28
Reserved	SS2		24
ESP2			20
Reserved	SS1		16
ESP1			12
Reserved	SS0		8
ESP0			4
Reserved	Previous Task Link		0

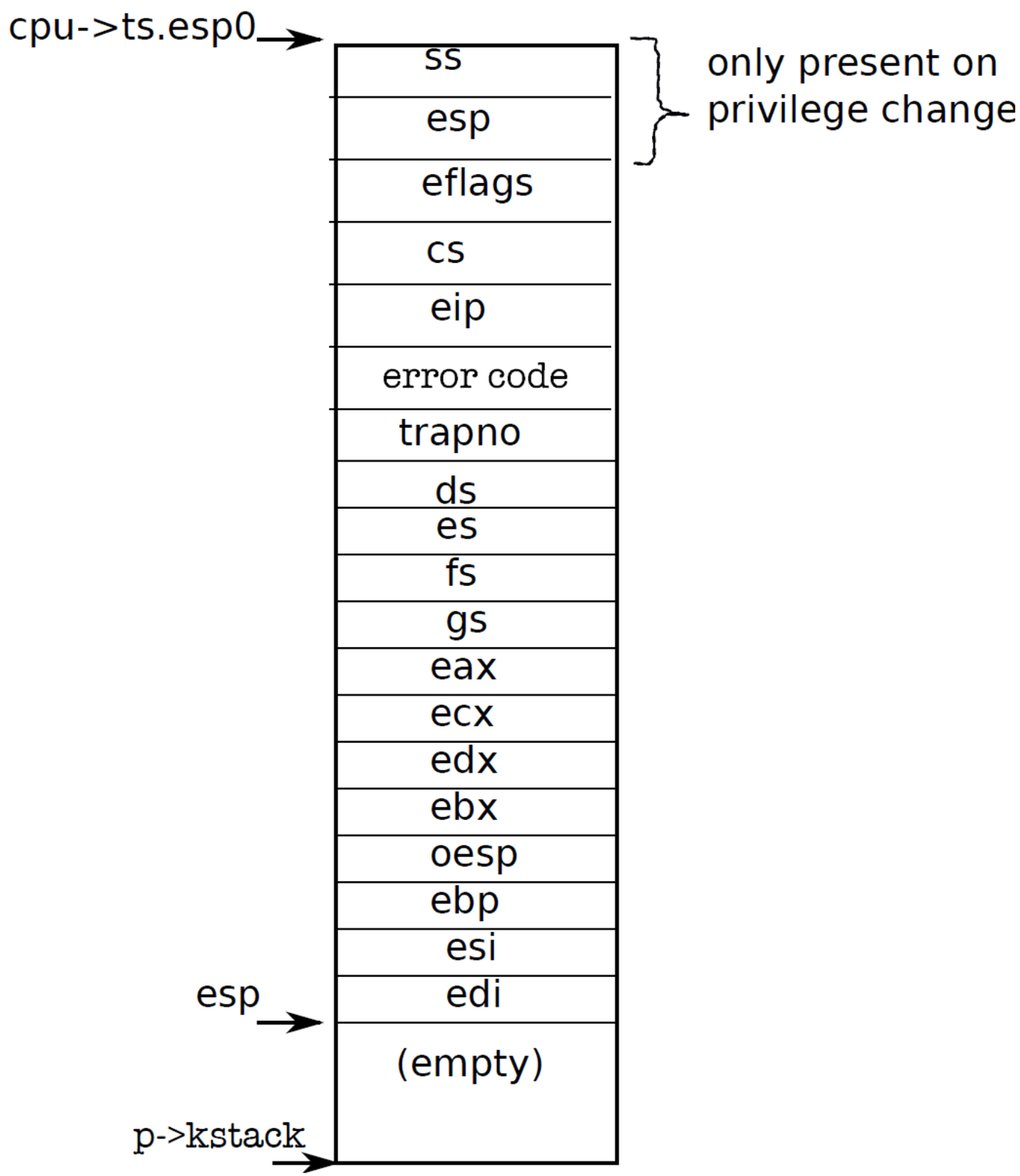
Reserved bits. Set to 0.

Figure 7-2. 32-Bit Task-State Segment (TSS)

switchuvm

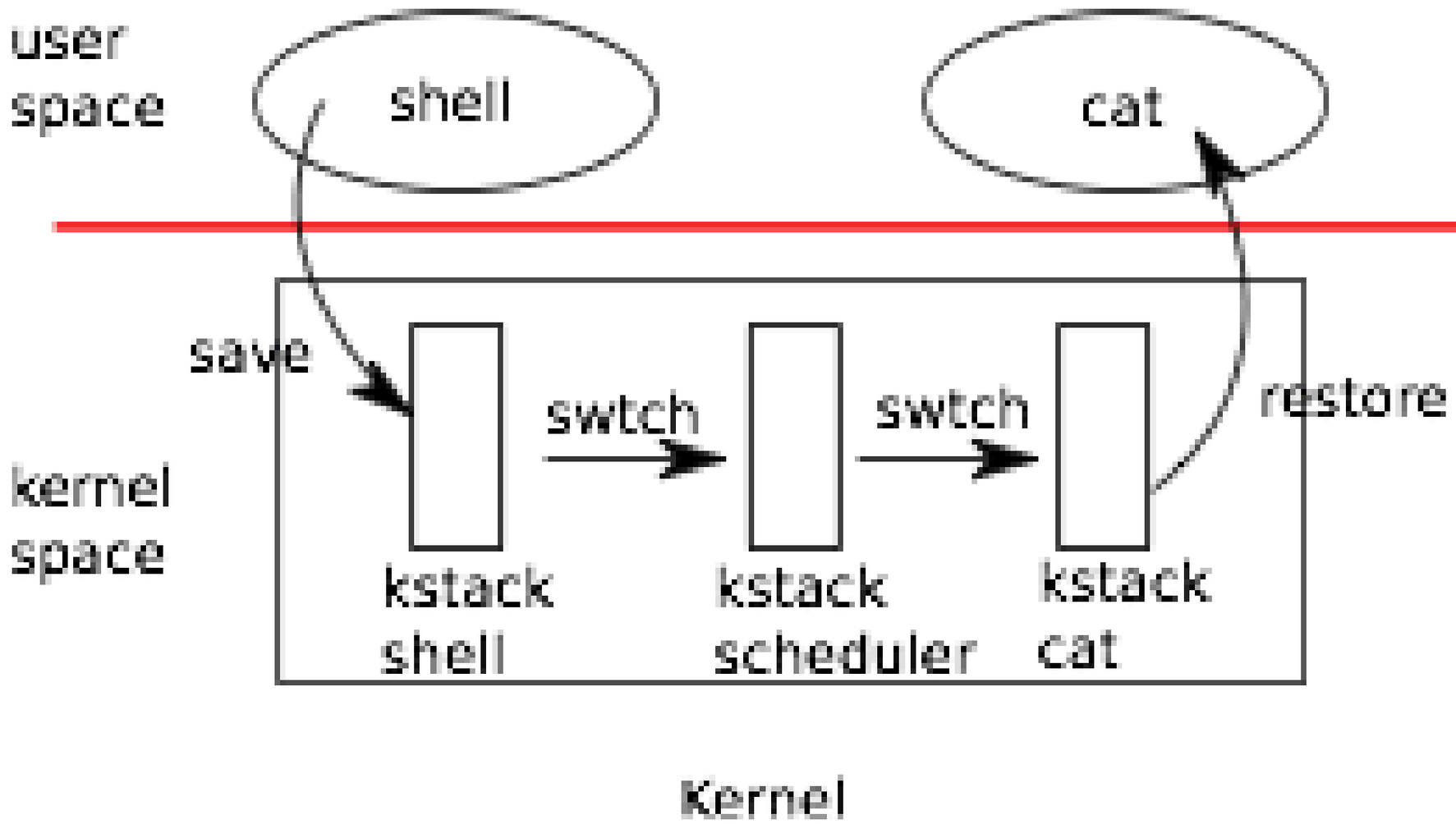
Set the Stack segment that instructs the hardware to execute system calls and interrupts on the process stack

```
// Switch TSS and h/w page table to correspond to process p.
void
switchuvm(struct proc *p)
{
  pushcli();
  cpu->gdt[SEG_TSS] = SEG16(STS_T32A, &cpu->ts, sizeof(cpu->ts)-1, 0);
  cpu->gdt[SEG_TSS].s = 0;
  cpu->ts.ss0 = SEG_KDATA << 3;
  cpu->ts.esp0 = (uint)proc->kstack + KSTACKSIZE;
  ltr(SEG_TSS << 3);
  if(p->pgdir == 0)
    panic("switchuvm: no pgdir");
  lcr3(v2p(p->pgdir));  // switch to new address space
  popcli();
}
```



igure 3-2. The trapframe on the kernel stack

swtch


user
space
shell
cat
save
swtch
swtch
restore
kernel
space
kstack
shell
kstack
scheduler
kstack
cat
Kernel

```
void
scheduler(void)
{
  struct proc *p;

  for(;;){
    // Enable interrupts on this processor.
    sti();

    // Loop over process table looking for process to run.
    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->state != RUNNABLE)
        continue;

      // Switch to chosen process.  It is the process's job
      // to release ptable.lock and then reacquire it
      // before jumping back to us.
      proc = p;
      switchuvm(p);
      p->state = RUNNING;
      swtch(&cpu->scheduler, proc->context);
      switchkvm();

      // Process is done running for now.
      // It should have changed its p->state before coming back.
      proc = 0;
    }
    release(&ptable.lock);

  }
}
```

swtch

- Does the actual work of switching between the kernel scheduler context and the kernel process context.
- Only needs to save a few registers :

```
struct context {
  uint edi;
  uint esi;
  uint ebx;
  uint ebp;
  uint eip;
};
```

- What about the others? What about eax?

What About the Others?

- Don't need to save segment registers because those are the same for all kernel contexts
- Don't need to save eax, ecx, edx – in the gcc calling convention, these are not assumed to persist across function calls
- So any code that calls swtch() will automatically preserve them

```
# Context switch
#
#   void swtch(struct context **old, struct context *new);
#
# Save current register context in old
# and then load register context from new.

.globl swtch
swtch:
  movl 4(%esp), %eax
  movl 8(%esp), %edx

  # Save old callee-save registers
  pushl %ebp
  pushl %ebx
  pushl %esi
  pushl %edi

  # Switch stacks
  movl %esp, (%eax)
  movl %edx, %esp

  # Load new callee-save registers
  popl %edi
  popl %esi
  popl %ebx
  popl %ebp
  ret
```

Load arguments into eax and edx
Notice do this before switching stacks below

```
# Context switch
#
#   void swtch(struct context **old, struct context *new);
#
# Save current register context in old
# and then load register context from new.

.globl swtch
swtch:
  movl 4(%esp), %eax
  movl 8(%esp), %edx

  # Save old callee-save registers
  pushl %ebp
  pushl %ebx
  pushl %esi
  pushl %edi

  # Switch stacks
  movl %esp, (%eax)
  movl %edx, %esp

  # Load new callee-save registers
  popl %edi
  popl %esi
  popl %ebx
  popl %ebp
  ret
```

Push the register state

```
struct context {
  uint edi;
  uint esi;
  uint ebx;
  uint ebp;
  uint eip;
};
```

```
# Context switch
#
#   void swtch(struct context **old, struct context *new);
#
# Save current register context in old
# and then load register context from new.

.globl swtch
swtch:
  movl 4(%esp), %eax
  movl 8(%esp), %edx

  # Save old callee-save registers
  pushl %ebp
  pushl %ebx
  pushl %esi
  pushl %edi

  # Switch stacks
  movl %esp, (%eax)
  movl %edx, %esp

  # Load new callee-save registers
  popl %edi
  popl %esi
  popl %ebx
  popl %ebp
  ret
```

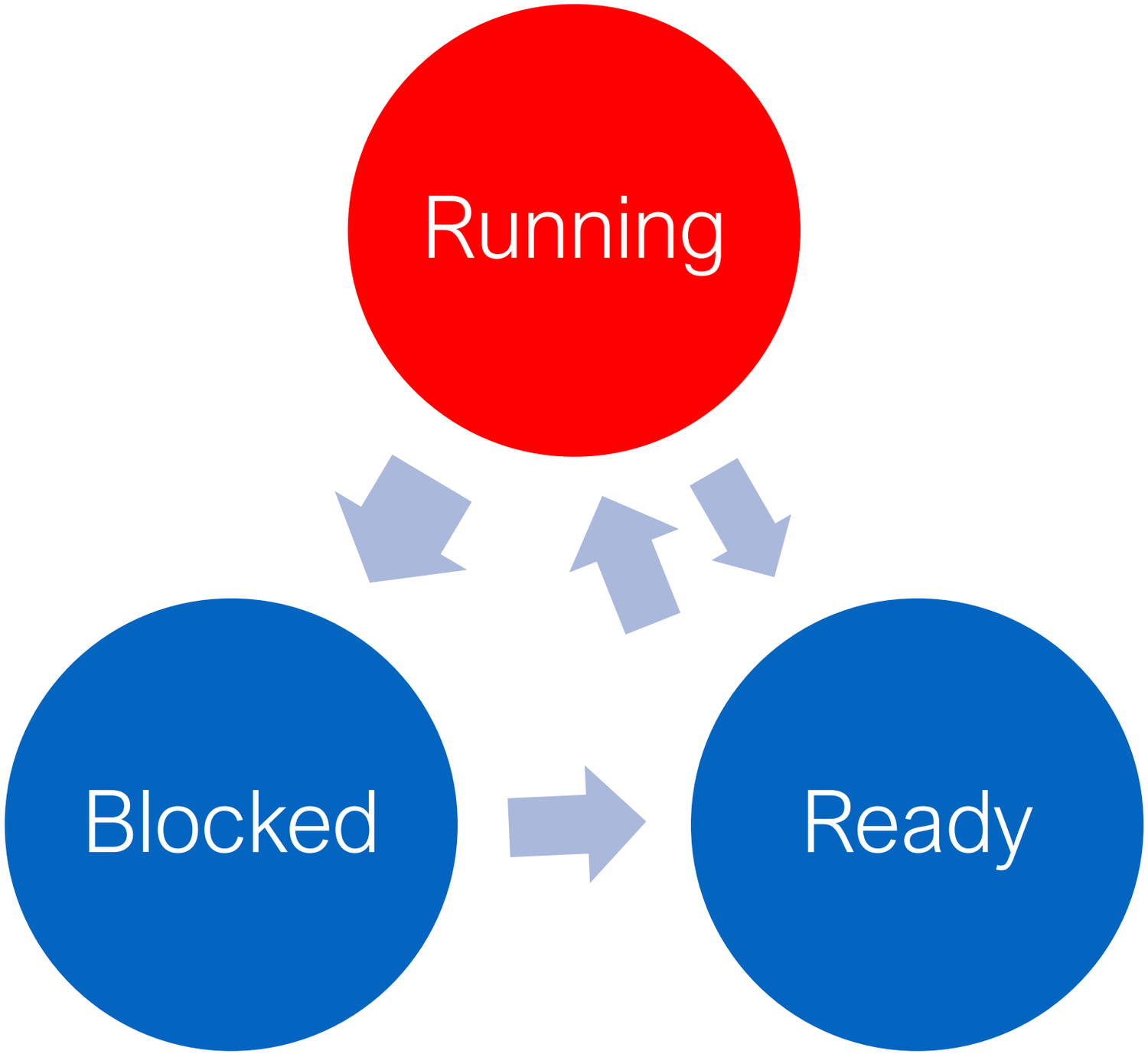
Note: EIP is *implicitly* saved by executing the call – call pushes the EIP of the next instruction onto the stack

```
struct context {
  uint edi;
  uint esi;
  uint ebx;
  uint ebp;
  uint eip;
};
```

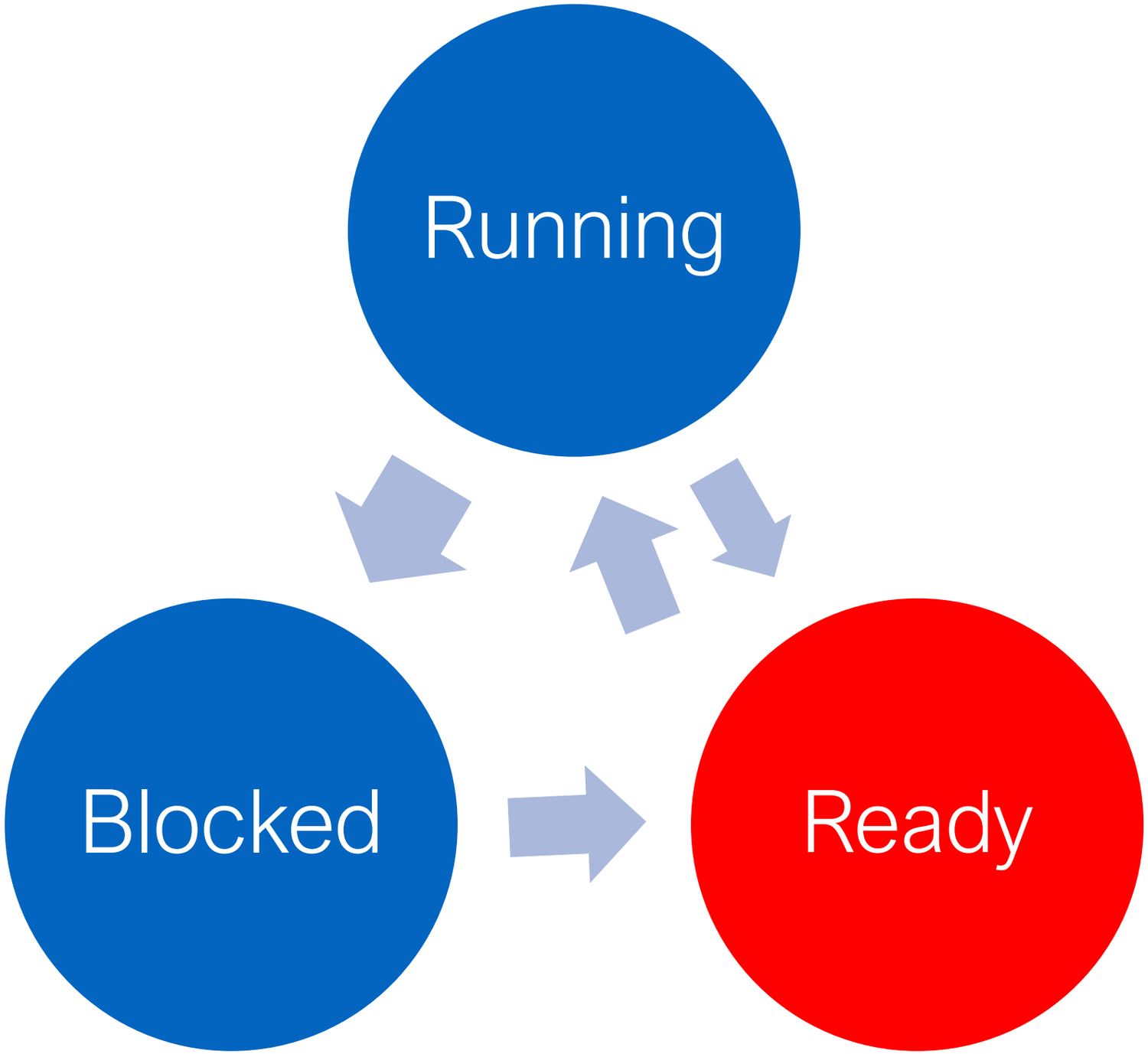
Context (proc.h)

```c
// Saved registers for kernel context switches.
// Don't need to save all the segment registers (%cs, etc),
// because they are constant across kernel contexts.
// Don't need to save %eax, %ecx, %edx, because the
// x86 convention is that the caller has saved them.
// Contexts are stored at the bottom of the stack they
// describe; the stack pointer is the address of the context.
// The layout of the context matches the layout of the stack in swtch.S
// at the "Switch stacks" comment. Switch doesn't save eip explicitly,
// but it is on the stack and allocproc() manipulates it.
struct context {
  uint edi;
  uint esi;
  uint ebx;
  uint ebp;
  uint eip;
};
```

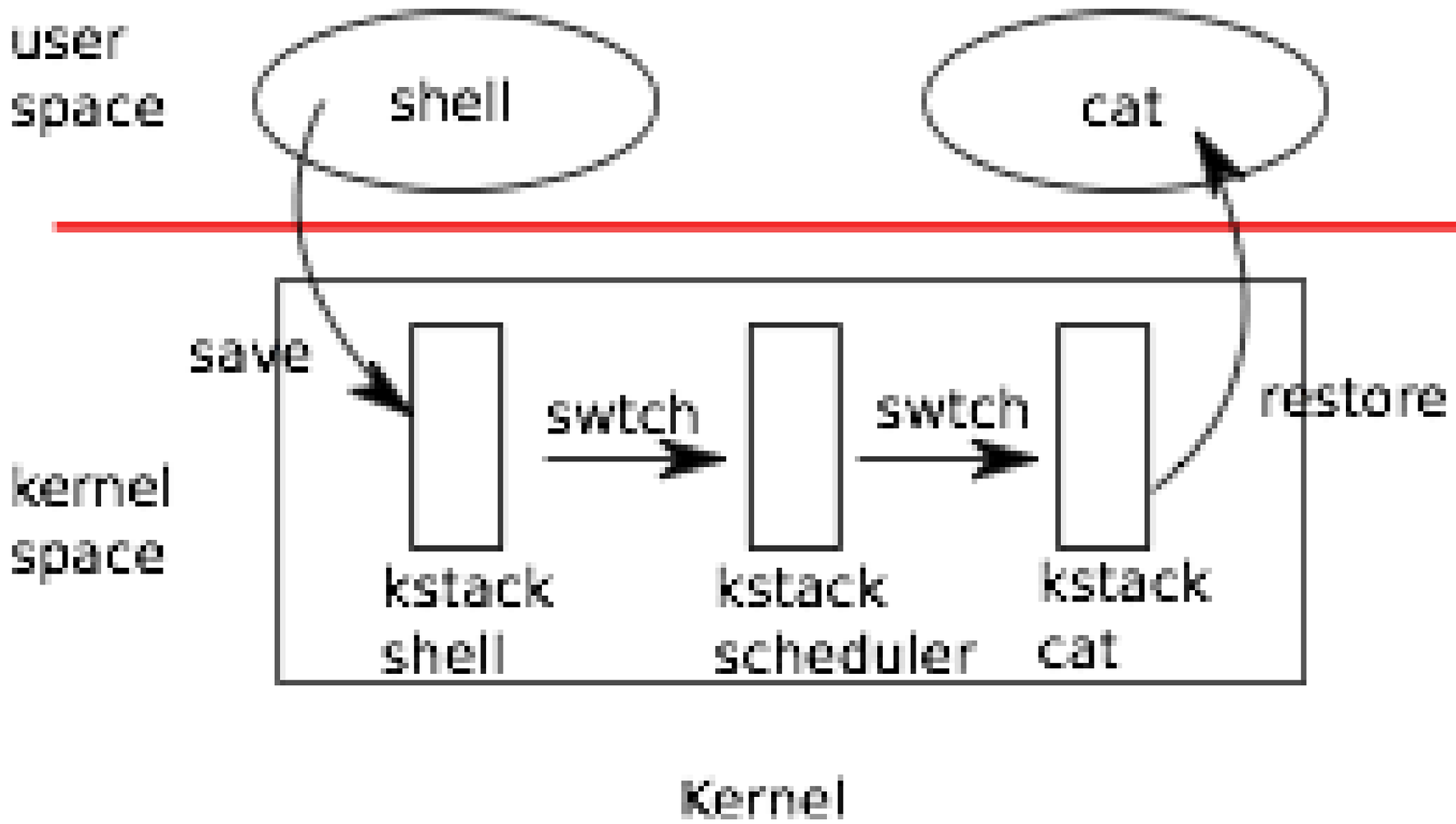
Now process is running



Switching Back(Finished Quantum)



swtch


user
space
shell
cat
save
swtch
swtch
restore
kernel
space
kstack
shell
kstack
scheduler
kstack
cat
Kernel

swtch-ing Back

- When a process "gives up" the CPU, yield() is called
- yield() makes the process runnable and then calls sched()
- Note: *not* the same as scheduler()

```
// Give up the CPU for one scheduling round.
void
yield(void)
{
  acquire(&ptable.lock);  //DOC: yieldlock
  proc->state = RUNNABLE;
  sched();
  release(&ptable.lock);
}
```

sched()

```
// Enter scheduler.  Must hold only ptable.lock
// and have changed proc->state.
void
sched(void)
{
  int intena;

  if(!holding(&ptable.lock))
    panic("sched ptable.lock");
  if(cpu->ncli != 1)
    panic("sched locks");
  if(proc->state == RUNNING)
    panic("sched running");
  if(readeflags()&FL_IF)
    panic("sched interruptible");
  intena = cpu->intena;
  swtch(&proc->context, cpu->scheduler);
  cpu->intena = intena;
}
```

sched()

```
// Enter scheduler.  Must hold only ptable.lock
// and have changed proc->state.
void
sched(void)
{
  int intena;

  if(!holding(&ptable.lock))
    panic("sched ptable.lock");
  if(cpu->ncli != 1)
    panic("sched locks");
  if(proc->state == RUNNING)
    panic("sched running");
  if(readeflags()&FL_IF)
    panic("sched interruptible");
  intena = cpu->intena;
  swtch(&proc->context, cpu->scheduler);
  cpu->intena = intena;
}
```

Where does this go?

```
void
scheduler(void)
{
  struct proc *p;

  for(;;){
    // Enable interrupts on this processor.
    sti();

    // Loop over process table looking for process to run.
    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
      if(p->state != RUNNABLE)
        continue;

      // Switch to chosen process.  It is the process's job
      // to release ptable.lock and then reacquire it
      // before jumping back to us.
      proc = p;
      switchuvm(p);
      p->state = RUNNING;
      swtch(&cpu->scheduler, proc->context);
      switchkvm();

      // Process is done running for now.
      // It should have changed its p->state before coming back.
      proc = 0;
    }
    release(&ptable.lock);

  }
}
```

Here